

Linux Home Lab Hardening & Compliance Audit Report

Prepared by Carmen Chai | September 2026

Executive Summary

This report documents a hands-on hardening and compliance audit performed on a fresh Ubuntu Server 26.04 LTS (ARM64) virtual machine, benchmarked against Lynis and CIS-aligned controls. The system's Lynis Hardening Index improved from **63/100** to **72/100** following remediation of SSH configuration, firewall policy, unnecessary services, account password policy, and audit logging. Each change is mapped below to its corresponding NIST 800-53 Rev. 5 control family, and evidence of implementation (configuration excerpts and live log output) is included in the accompanying repository.

Scope & Methodology

Environment: Ubuntu Server 26.04 LTS (ARM64), virtualized via UTM on Apple Silicon

Benchmark tool: Lynis 3.1.6

Framework mapping: NIST 800-53 Rev. 5

A baseline scan was performed immediately after OS installation, prior to any configuration changes. Findings were remediated in the following order: SSH daemon configuration, host-based firewall policy, unnecessary running services, user account and password aging policy, and audit logging (auditd). A final scan was performed after remediation to measure improvement.

Results

Metric	Before	After
Lynis Hardening Index	63 / 100	72 / 100
Suggestions flagged	45	Reduced across SSH, firewall, services, accounts, and audit categories

Findings & Control Mapping

Area	Change Made	NIST 800-53 Control
SSH Access	Disabled TCP/X11/agent forwarding, tightened session limits, verbose logging, reduced max auth tries	AC-3, IA-2
Firewall	Enabled ufw with default-deny incoming, explicit allow for SSH only	SC-7
Services	Disabled ModemManager (unused hardware service, high exposure score)	CM-7
User Accounts	Enforced password aging (max 90 / min 7 / warn 14 days), tightened umask to 027	AC-2, AC-6
Audit Logging	Installed and configured auditd; rules watch /etc/passwd, /etc/shadow, sudoers, sshd_config	AU-2, AU-12

Verification Evidence

Configuration changes were not assumed to be effective based on file edits alone. Each control was verified against live system state:

SSH: Effective configuration confirmed via `sshd -T`, cross-checked against all eight intended settings.

Firewall: Confirmed via `ufw status verbose` showing active status, default-deny incoming policy, and explicit SSH allow rule.

Audit Logging: A live test was performed by editing a watched file (`sshd_config`) and confirming the event was captured via `ausearch -k sshd_config_changes`, producing a logged event with correct timestamp, process (`nano`), and file path.

Sample Audit Log Evidence

```
type=SYSCALL msg=audit(1788927063.442:448): ... comm="nano" exe="/usr/bin/nano"
key="sshd_config_changes"
```

Key Takeaways

The `auditd` rules failed to load silently after initial configuration — `systemctl status` reported the service as active, but the real cause only surfaced after tracing the actual audit log and finding a stray line that had been accidentally written into the rules file instead of executed as a command. This reinforced that a service reporting "active" does not guarantee its configuration took effect — actual behavior must be verified, not just service status.

At scale, this process would be automated with a configuration management tool such as Ansible, with a validation step (e.g. `sshd -t`, `augenrules --check`) built in before any service restart, to catch errors before they cause downtime across a fleet of machines.